Homework Sets

15–20 minutes a week: Core, Challenge and Creative

Tip: How homework works

Do the **Core** task first (about 10 minutes). **Challenge** and **Creative** are optional extras for when you want more. Finish with the self-check list. If you get stuck for more than 5 minutes, write down what you tried and bring the question next week: that is a great result too.

Contents

Week 1: Save Points 2
Week 2: Objects: Blueprints and Things 4
Week 3: Families of Objects 6
Week 4: Searching and How Fast Is Fast? 8
Week 5: Sorting, Stacks and Queues 10
Week 6: Review and Checkpoint 12
Week 7: Grids, Recursion and Fractal Art 14
Week 8: Data Detectives 16
Week 9: How Do Machines Learn? 18
Week 10: Project Studio 1 20
Week 11: Project Studio 2 22
Week 12: Showcase and Graduation 24

Week 1 Level 3: Advanced

Save Points

15–20 minutes. Do Core first; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): Inventory Review + one commit (about 10 minutes)

1. Open `week01/hw-inventory-review/starter.py` in Thonny. A game backpack is a list like `["gem", "key", "gem"]`.
2. Finish the three TODOs: `count_item`, `remove_all` (make a **new** list) and `most_common`.
3. Open `check_my_work.py` and press Run until every check passes.
4. Git practice: copy your finished `starter.py` into `Documents/git-quest`, then in Terminal:

>_ Terminal

```
cd ~/Documents/git-quest
git add starter.py
git commit -m "Add inventory review"
git log --oneline
```

How many commits does your `git-quest` have now? _____

★★ Challenge (want more?): Message makeover

Rewrite each unhelpful message so that a friend would understand it. Start with a verb.

Unhelpful	Your better message
`stuff` (made the hero jump higher)	
`asdf` (added a sound when you score)	
`fix` (score no longer goes below 0)	

Then use your Commit Coach from class to test your three messages. Do they all say “Great message!”?

Creative (make it yours): A badge with a history

Change `hero_badge.py` in your `git-quest` folder at least 3 times: new colors from the palette, a bigger star, a second ring, your own hero name (made up, never your real name). Commit after **every** change. Then draw your timeline below: one circle per commit with its message.

Self-check: tick each box when it is true

- ☐ All checks pass in `hw-inventory-review`.
- ☐ My `git log --oneline` shows at least 4 commits.
- ☐ Every message starts with a verb and says what changed.
- ☐ My git name is a nickname and my email ends in `@example.com`.
- ☐ There are no passwords or real names anywhere in my repository.

Week 2 Level 3: Advanced

Objects: Blueprints and Things

15–20 minutes. Do Core first; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): Coin Purse (about 10 minutes)

1. Open `week02/hw-coin-purse/starter.py` in Thonny. A `CoinPurse` object keeps a player's coins.
2. Finish `add` (ignore `0` or negative amounts), `can_afford` and `spend` (only pay if you can afford it; return `True` or `False`).
3. Run it. The last line should be `Found 7 coins! Now: 9 coins.`
4. Open `check_my_work.py` and press Run until all checks pass.
5. Copy your file into `Documents/git-quest` and commit it with a helpful message.

Predict: `purse = CoinPurse(10)`, then `purse.spend(4)`, then `purse.spend(7)`. What does the second `spend` return, and how many coins are left? _____

★★ Challenge (want more?): Design a class on a card

Choose a thing from a game: a spaceship, a plant, a door, a monster. Fill in its class card. Then, if you have time, write it in Python with `__init__` and two methods, and make two objects.

Class name	
Attributes (facts)	
Methods (actions)	
Two objects	

Creative (make it yours): Make the bubbles yours

Add a new method to your `Bubble` class from class, for example `pop(self, t)` that draws a ring of 8 small dots around the bubble, or `rainbow(self, t)` that draws 3 smaller dots inside it in different palette colors. Use it on some of the bubbles. Sketch your picture here:

Self-check: tick each box when it is true

- ☐ All checks pass in `hw-coin-purse`.
- ☐ Every method I wrote has `self` as its first parameter.
- ☐ I can explain the difference between a class and an object.
- ☐ I committed my work in `git-quest` with a helpful message.

Week 3
Families of Objects
Level 3: Advanced

15–20 minutes. Do Core first; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): Hero Party (about 10 minutes)

1. Open `week03/hw-hero-party/starter.py` in Thonny. `Hero` (the parent) and `Healer` (a child) are done: read them first.
2. Copy the `Healer` pattern to finish `Builder` (power `"building"`, action "builds a bridge") and `Explorer` (power `"exploring"`, action "maps the cave").
3. Finish `Party.take_turn`: every hero acts once (composition: the party *has* heroes).
4. Run it, then open `check_my_work.py` and press Run until all checks pass. Commit your work in `git-quest`.

Is a or has a? Write "is a" or "has a" in each gap.

- A Healer _____ Hero. A Party _____ Heroes. A Builder _____ Hero.

★★ Challenge (want more?): Predict the output

Without running it, write what this prints. Then type it in and check.

```python
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        return self.name + " makes a sound"

class Owl(Animal):
    def speak(self):
        return self.name + " says hoo"

class Fox(Animal):
    pass

for a in [Owl("Olu"), Fox("Fen"), Animal("Ash")]:
    print(a.speak())
```

Creative (make it yours): Your own hero

Invent a new hero class (a child of `Hero`) with its own power and action, for example a `Musician` who "plays a brave song" or a `Gardener` who "grows a vine ladder". Add two of them to the party. Give your heroes names from many places and cultures. Draw your hero here:

Self-check: tick each box when it is true

- ☐ All checks pass in `hw-hero-party`.
- ☐ My child classes use `super().__init__`.
- ☐ I can explain the difference between "is a" and "has a".
- ☐ I committed my work in `git-quest` with a helpful message.

Week 4 Level 3: Advanced

Searching and How Fast Is Fast?

15–20 minutes. Do Core first; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): Growth Table (about 10 minutes)

1. Predict first. How many steps does each search need in the worst case? Fill in the empty boxes *before* you run any code.

Items	Linear	Binary
10		
100		
1000		
1 000 000		

2. Code. Open `week04/hw-growth-table/starter.py`. Finish `linear_worst(n)` and `halving_steps(n)`: keep doing `n = n // 2` and count, until n is 0.

3. Check. Run it and compare with your predictions. Then open `check_my_work.py` and press Run. Commit your work in `git-quest`.

★★ Challenge (want more?): Trace it by hand

Use binary search to find **9** in `[1, 3, 5, 7, 9, 11, 13]`. Fill in one row per step.

step	low	high	mid	items[mid]	so…
1					
2					
3					

How many steps did it take? _____ How many would linear search take? _____

Creative (make it yours): Draw the race

Draw a bar chart of your growth table on paper or with turtle: one pair of bars for each list size. Linear bars get too tall very quickly! How could you draw them so they still fit on the page? (Idea: let 1 cm stand for more and more steps, and write the real number on each bar.)

Self-check: tick each box when it is true

- ☐ I wrote my predictions before running the code.
- ☐ All checks pass in `hw-growth-table`.
- ☐ I can explain why binary search grows so slowly.
- ☐ I committed my work in `git-quest`.

Week 5
Sorting, Stacks and Queues
Level 3: Advanced

15–20 minutes. Do Core first; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): Turn Queue (about 10 minutes)

Four friends share one game. A queue makes it fair: the player at the front plays, then goes to the back of the line.

1. Open `week05/hw-turn-queue/starter.py` in Thonny.
2. Finish `take_turn`: `player = queue.popleft()`, then `queue.append(player)`, then return the player.
3. Finish `leave`: remove a player who has to go home, and return `True` (or `False` if they were not in the queue).
4. Run it, then open `check_my_work.py` and press Run. Commit your work in `git-quest`.

Predict: the queue is Priya, Sam, Kai, Zoe. Who plays turns 1 to 6? _____

★★ Challenge (want more?): Stack or queue?

Write **S** (stack) or **Q** (queue) next to each one, and say why in a few words.

Situation	S or Q	Why
The back button in a web browser		
Songs waiting in a playlist		
Printing 3 documents sent one after another		
A pile of homework sheets, newest on top		

Creative (make it yours): Design a fair leaderboard

Sketch a leaderboard screen for a game you like. Use nicknames only. Include at least one thing that celebrates effort, not only winning (for example “most improved”, “personal best”, or “longest streak

of helping”).

Self-check: tick each box when it is true

- ☐ All checks pass in `hw-turn-queue`.
- ☐ I can explain “last in, first out” and “first in, first out”.
- ☐ My leaderboard sketch uses nicknames only.
- ☐ I committed my work in `git-quest`.

Week 6
Review and Checkpoint
Level 3: Advanced

15–20 minutes. Do Core first; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): Piggy Bank Bug Hunt (about 10 minutes)

1. Open `week06/hw-bug-bank/starter.py`. Read the expected output at the top of the file, then press Run. What is different?
2. Find the 3 bugs. Use the debugger: Debug, Step over, Step into, and watch `self.coins` and `self.log` in the Variables view.
3. Fill in your bug log, then open `check_my_work.py` and press Run until all checks pass. Commit the fixed file in `git-quest`.

Bug	Line	What was wrong	How I fixed it
1			
2			
3			

★★ Challenge (want more?): Trace table

Without running it, fill in the trace table for this code. Then check with the debugger.

```
stack = []
for word in ["red", "green", "blue"]:
    stack.append(word)
top = stack.pop()
```

After line	stack	top
append “red”		
append “green”		
append “blue”		
`top = stack.pop()`		

Creative (make it yours): Bug comic

Draw a 3-panel comic about a bug you met this term: panel 1, the bug appears; panel 2, the hunt (what strategy did you use?); panel 3, the fix. Bugs can be cute!

Self-check: tick each box when it is true

- ☐ All checks pass in `hw-bug-bank`.
- ☐ I used the debugger at least once at home.
- ☐ I changed one thing at a time and ran the program after each change.
- ☐ I committed my fixed file in `git-quest`.

Week 7 Level 3: Advanced

Grids, Recursion and Fractal Art

15–20 minutes. Do Core first; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): Treasure map (about 10 minutes)

Open `3-advanced/exercises/week07/hw-treasure-map/starter.py` in Thonny and press F5. The map is a grid: ~ water, ^ mountain, . grass, T treasure, S start.

1. Finish `count_tiles(grid, tile)`. Hint: a list has `.count()`, e.g. `["a", "b", "a"].count("a")` is 2.
2. Finish `neighbors(grid, row, col)`: the cells up, down, left and right – but never off the edge of the map.
3. Run `check_my_work.py` until every check passes.

How many neighbors does a corner cell have? _____ A cell in the middle? _____

★★ Challenge (want more?): Koch snowflake

Open `week07/hw-koch-snowflake/starter.py`. A Koch line of depth 0 is a straight line; a Koch line of depth n is **four** Koch lines of depth $n-1$, each a third as long, with turns of 60, 120, 60 degrees making a bump: `___/\___`. Replace the straight line in the recursive case and run it.
Fill in: depth 1 = 4 lines per side, depth 2 = _____, depth 3 = _____. What is the base case?

Creative (make it yours): My own fractal

Change the fractal tree from class (ANGLE, SHRINK, colors, three branches, a leaf dot in the base case) or draw a recursive pattern of your own: squares inside squares, a spiral, circles on circles. Add a comment at the top: `# Based on: ...` to credit the idea. Sketch it here:

Self-check: tick each box when it is true

- ☐ I can point to the base case and the recursive case in a recursive function.
- ☐ I know `grid[row][col]` means row first, then column, both from `0`.
- ☐ My `neighbors` function never returns a cell off the map.
- ☐ If I see `RecursionError`, I check the base case first.

Week 8 Level 3: Advanced

Data Detectives

15–20 minutes. Do Core first; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): Anonymize a survey (about 10 minutes)

Open `3-advanced/exercises/week08/hw-survey-privacy/starter.py` and press F5. The file `snack_survey.csv` is made up, but it has a `nickname` and a `club` column. Together they could show exactly who answered.

1. Finish `anonymize(rows, keep)`: each new row gets an `id` (R1, R2, ...) and only the columns in `keep`.
2. Run again. The first row should be `{'id': 'R1', 'favorite_snack': 'apple slices', 'minutes_reading': '20'}`.
3. Run `check_my_work.py`.

Why is `club` risky even without a name?

_____

★★ Challenge (want more?): Save the safe file

Finish `save_csv(rows, filename)` with `csv.DictWriter`: open the file with `"w"` and `newline=""`, write the header, then the rows. Run the program and open `snack_survey_anonymous.csv` in Thonny (**File** → **Open**, choose "All files"). Which snack is the most popular?

_____

Creative (make it yours): Design a kind, safe survey

Plan a 4-question survey for your family about a fun topic (favorite board game, best pizza topping, dream pet). Rules: start with a consent sentence ("You can skip any question"); no names, ages, addresses or schools; at least one question with a number answer you could average.

Self-check: tick each box when it is true

- ☐ I know that CSV values arrive as text and must be converted to numbers.
- ☐ I can explain why we skip empty answers instead of counting them as `0`.
- ☐ My anonymous rows have IDs and no nickname or club.
- ☐ I know to ask for consent and collect only what I need.

Week 9 Level 3: Advanced

How Do Machines Learn?

15–20 minutes. Do both Core parts; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): Part A: Fruit detective (about 10 minutes)

Open `3-advanced/exercises/week09/hw-fruit-detective/starter.py`. One card in `TRAINING` has the wrong label: a lemon was labeled "apple". With **k = 1** the classifier copies the single closest card. With **k = 3** the three closest cards **vote**.

1. Finish `k_nearest(training, point, k)`: sort the cards by distance (`sorted(..., key=...)` from week 5) and keep the first k.
2. Finish `vote(labels)` with a counting dictionary.
3. Run it and `check_my_work.py`. k = 1 says: _____ k = 3 says: _____

★ Core (everyone): Part B: Finish your project pitch (Milestone 1, about 5 minutes)

Finish the pitch card in workbook Exercise 9.4. Check it against the Project brief: two or more classes (one with inheritance or composition), a dictionary/stack/queue/grid, a named algorithm, a saved file. Bring it to the next session.

★★ Challenge (want more?): How big should k be?

In the fruit detective, try k = 1, 3, 5 and 13 for the mystery fruit (5, 4) and for (2, 8). What happens when *every* card votes (k = 13)? Why is a very big k a bad idea too?

Creative (make it yours): Spot the AI

This week, notice three places where AI might be at work (video suggestions, photo filters, voice assistants, game opponents…). For one of them, write: what are its *examples* (training data)? Could it be unfair to someone? Do not type anything personal into any AI tool to find out!

Self-check: tick each box when it is true

- ☐ I can explain the difference between rules and learning from examples.
- ☐ I know why we test on cards the program did not learn from.
- ☐ I can explain how bad or unbalanced examples lead to unfair predictions.
- ☐ I never share personal information with a chatbot.
- ☐ My project pitch card is complete.

Week 10 Level 3: Advanced

Project Studio 1

15–20 minutes of project work. Do Core first; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): Build your core (Milestone 2)

Work on your project in `Documents/coding-projects/<your project>`.

1. Pick **one** class from your CRC cards and make its `__init__` and one method really work.
2. Run it. When it works, commit in Terminal:

>_ Terminal

```
git add .
git commit -m "Make Player move one cell"
```

3. Repeat for another small step. Aim for **3 or more new commits** this week.

Copy your new commit messages from `git log --oneline`:

★★ Challenge (want more?): Your first test

Create `test_<your project>.py` next to your code and write one test for the method you just finished (look at the example in workbook chapter 11). Run it in Thonny with **Tools** → **Open system shell...** and `python -m pytest -q`. Does it pass? Break the code on purpose: does it fail?

Creative (make it yours): Title screen

Design the first thing a player sees: the name of your project, the controls, and a picture made with turtle. Use the Okabe–Ito palette and write the controls in text (not color alone). Sketch it here, then code it if you have time.

Self-check: tick each box when it is true

- ☐ My project folder is a git repository (`git log` works there).
- ☐ At least one class really works, not just `pass`.
- ☐ I made at least 3 commits with messages that start with a verb.
- ☐ Everything in my project is mine or allowed by its license, and listed for my Credits.

Week 11
Project Studio 2
Level 3: Advanced

15–20 minutes. Finish Milestone 4 and get ready for the showcase.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): Finish Milestone 4 and prepare your talk

1. Finish your `README.md`: title, how to run, controls, how it works (classes, data structure, algorithm, saved file), tests, credits. Commit it.
2. Fix one more "must fix" or "should fix" bug from your playtest cards. Commit it.
3. Run all your tests: `python -m pytest -q`. Number passing: _____ (goal: 5 or more)
4. Fill in the showcase talk card (workbook Exercise 12.1) and practice once out loud with a timer: 2 minutes.

★★ Challenge (want more?): An edge-case hunt

Think like a sneaky player: what could break your project? (Pressing keys very fast, walking into a corner, an empty save file, the last life.) Write a test for one of them. If it fails, fix the code. Commit with a message like `Test and fix empty high-score file`.

Creative (make it yours): Box art

Design the box or poster for your project as if it were sold in a shop: the title, one exciting sentence, a picture, and "Made by" with your *nickname*. Keep it at home – no posting online.

Self-check: tick each box when it is true

☐ I have 5 or more passing tests, including an edge case.

☐ My README explains my classes, data structure and algorithm.

- ☐ My project passes the accessibility checklist (not color alone, readable text, controls on screen).
- ☐ I have 6 or more commits and know my talk's Problem, Choice, Why and Next time.

Week 12 Level 3: Advanced

Showcase and Graduation

15–20 minutes. Your last homework of Level 3 – congratulations!

Name (nickname is fine): _____ Date: _____

★ Core (everyone): A letter to future-you

Write a short letter to yourself to open in one year. Include: what you built, the bug you are proudest of fixing, a piece of advice for when coding gets hard, and one thing you want to be able to build by then.

★★ Challenge (want more?): Keep coding: pick from the menu

Choose one item and plan your first step (with a parent, offline-first):

- **Version 2:** add a level editor, a second enemy type or a settings file to your project. Commit each step.
- **New algorithm:** write a maze generator using recursion, or sort a high-score table by two keys.
- **New data:** run a (consent-first, anonymous) family survey and chart it.
- **New AI idea:** train the monster sorter on your own made-up cards with three labels.
- **Teach it:** show a younger friend or relative how to draw a fractal tree.

My choice and first step:

Creative (make it yours): Your developer badge

Design a badge for yourself as a Level 3 graduate: a symbol that stands for your project, your nickname, and three skills you are proud of (e.g. “recursion”, “git”, “tests”).

Self-check: tick each box when it is true

- ☐ I gave my 2-minute talk and explained one design decision.
- ☐ I gave kind, specific feedback to others.
- ☐ I know how to look for coding communities safely, together with a parent.
- ☐ I have a plan for what to learn next.